

AI Smart Campus Assistance System

An Integrated AI-Based Request Handling and Navigation System

Course: Artificial Intelligence

Semester: Spring 2026

Submitted By:

Hajra Haseeb (23F-0789)

Sara Shakeel (23F-0617)

Salma Zahid (23F-0852)

Submitted To:

Mr. Muzaffar Abbas

Department:

Computer Science

1. Project Overview

The **AI Smart Campus Assistance System** is a CLI-based intelligent system that processes structured campus service requests using Artificial Intelligence techniques.

The system helps users with:

- Campus navigation
- Eligibility checking
- Slot scheduling
- Priority prediction
- Full service request handling

The project integrates multiple AI concepts to automate campus assistance tasks.

2. AI Concepts Used

2.1 Search Algorithms

Used for route finding between campus locations.

Algorithms:

- BFS
- UCS
- A*

Example:

Hostel → Cafeteria → AI Lab

2.2 Logic / Knowledge Base

Used for checking eligibility and rule-based decisions.

Example:

If a user is enrolled in AI, they can access AI Lab.

2.3 Constraint Satisfaction Problem (CSP)

Used for scheduling and assigning slots/rooms while avoiding conflicts.

Example:

Assigning the nearest available lab slot.

2.4 Artificial Neural Network (ANN)

Used for predicting request urgency.

Models:

- Perceptron
- MLP

Output:

- Low
- Normal
- High
- Urgent

3. Request Types

The system supports five request types:

Navigation Only

Returns route guidance.

Eligibility Check

Checks whether request is allowed.

Booking / Scheduling

Assigns room and slot.

Urgent Service Request

Predicts urgency and schedules.

Full Service Request

Combines all modules.

4. System Workflow

The system follows this pipeline:

User Input → Preprocessing → Router → AI Modules → Final Response

Preprocessing

Validates and standardizes input.

Router

Selects required modules.

Module Execution

Runs Search / Logic / CSP / ANN.

5. Code Snippets

Router logic:

```
request_type = request.get("request_type", "")
```

```
# Default all flags to False
```

```
needs_ann    = False
```

```
needs_logic  = False
```

```
needs_csp    = False
```

```
needs_search = False
```

```
pipeline     = []
```

```
if request_type == "Navigation_Only":
```

```

    pipeline    = ["Search"]
    needs_search = True

elif request_type == "Eligibility_Check":
    pipeline    = ["Logic_KB"]
    needs_logic = True

elif request_type == "Booking_or_Scheduling":
    pipeline    = ["Logic_KB", "CSP"]
    needs_logic = True
    needs_csp   = True
    # Search is optional – include if user provided a location
    if request.get("current_location"):
        pipeline.append("Search")
        needs_search = True

elif request_type == "Urgent_Service_Request":
    pipeline    = ["ANN", "Logic_KB", "CSP"]
    needs_ann   = True
    needs_logic = True
    needs_csp   = True
    if request.get("current_location"):
        pipeline.append("Search")
        needs_search = True

```

```

elif request_type == "Full_Service_Request":
    pipeline = ["ANN", "Logic_KB", "CSP", "Search"]
    needs_ann = True
    needs_logic = True
    needs_csp = True
    needs_search = True

else:
    # Unknown request type – reject immediately
    pipeline = []

router_output = {
    "request_id": request.get("request_id", ""),
    "selected_pipeline": pipeline,
    "needs_ann": needs_ann,
    "needs_logic": needs_logic,
    "needs_csp": needs_csp,
    "needs_search": needs_search,
    "valid": bool(pipeline),
}

return router_output

```

ANN encoding:

```
import math
```

```
# — Encoding maps
```

```
ROLE_ENCODING = {  
    "student": 0,  
    "instructor": 1,  
    "staff": 2  
}
```

```
REQUEST_TYPE_ENCODING = {  
    "AI_Lab_Support": 0,  
    "Viva_Scheduling": 1,  
    "Access_Request": 2,  
    "Maintenance": 3,  
    "Emergency_Help": 4  
}
```

```
FEATURE_ORDER = [  
    "Role", "RequestType", "Severity",  
    "TimeSensitivity", "CrowdLevel", "Distance", "Eligibility"  
]
```

Campus distance table from common starting locations to AI_Lab (approx hops)

```
CAMPUS_DISTANCES = {  
    "Hostel": 4, "Cafeteria": 3, "Science_Block": 2, "AI_Lab": 0,  
    "Library": 2, "Seminar_Room": 3, "Exam_Hall": 4, "Student_Services": 4,  
    "Admin_Block": 4, "Main_Gate": 4, "Parking": 4, "Medical_Center": 5,  
    "Bus_Stop": 6  
}
```

```
def build_feature_vector(request: dict) -> list:
```

```
    """
```

Convert a standardized request dictionary into a numeric feature vector.

Feature order: [Role, RequestType, Severity, TimeSensitivity,

CrowdLevel, Distance, Eligibility]

Parameters:

request (dict): Standardized request object.

Returns:

list: Numeric feature vector of length 7.

```
    """
```

```
    role_enc = ROLE_ENCODING.get(request.get("role", "student"), 0)
```

```
    rt_enc = REQUEST_TYPE_ENCODING.get(request.get("category",  
"AI_Lab_Support"), 0)
```

```
    severity = float(request.get("severity", 5))
```

```
    time_sens = float(request.get("time_sensitivity", 5))
```

```

crowd    = float(request.get("crowd_level", 5))
distance = float(CAMPUS_DISTANCES.get(request.get("current_location",
"Hostel"), 4))

eligibility = 1.0 if request.get("eligibility_claim", True) else 0.0

```

MAIN LOGIC

```
def execute_pipeline(request: dict) -> dict:
```

```
    """
```

Run the full AI pipeline based on the router's decision.

Parameters:

request (dict): Validated and standardized request object.

Returns:

dict: Final response object.

```
    """
```

```
# — Step 1: Request Router —————
```

```
router_output = route_request(request)
```

```
display_router_output(router_output)
```

```
if not router_output["valid"]:
```

```
    return {
```

```
        "request_id": request.get("request_id", ""),
```

```
        "decision": "rejected",
```

```
        "message": "Unknown request type. Request rejected by router.",
```

```
        "priority": {},
```

```
"eligibility": {},  
"assignment": {},  
"route":    {},  
}
```

```
ann_output  = None
```

```
logic_output = None
```

```
csp_output  = None
```

```
search_output = None
```

```
# — Step 2: ANN Priority (if needed) —————
```

```
if router_output["needs_ann"]:
```

```
    print("\n[→] Running ANN Priority Module...")
```

```
    ann_output = predict_priority(request)
```

```
    display_ann_output(ann_output)
```

```
# — Step 3: Logic / KB (if needed) —————
```

```
if router_output["needs_logic"]:
```

```
    print("\n[→] Running Logic / Knowledge Base Module...")
```

```
    logic_output = run_logic_check(request)
```

```
    display_logic_output(logic_output)
```

```
# Gatekeeper: stop here if not allowed
```

```
if not logic_output.get("allowed", True):
```

```
    print("\n[X] Request rejected by Logic/KB. Pipeline halted.")
```

```
    return build_final_response(request, ann_output, logic_output)
```

— Step 4: CSP Scheduler (if needed) —————

if router_output["needs_csp"]:

 print("\n[→] Running CSP Scheduler Module...")

 csp_output = run_csp(request)

 display_csp_output(csp_output)

 # If CSP could not schedule, stop here

 if csp_output.get("decision") == "rejected":

 print("\n[X] CSP could not find a valid assignment. Pipeline halted.")

 return build_final_response(request, ann_output, logic_output, csp_output)

— Step 5: Search & Navigation (if needed) —————

if router_output["needs_search"]:

 print("\n[→] Running Search & Navigation Module...")

 # Determine source and destination

 source = request.get("current_location", "")

 dest = request.get("destination", "")

 # After CSP, destination might come from CSP assignment

 if not dest and csp_output:

 dest = csp_output.get("destination", "")

 if not source or not dest:

 print(" [!] Source or destination missing – skipping Search.")

```

else:

    # Decide graph type (weighted for full/urgent, unweighted for simple nav)
    request_type = request.get("request_type", "")
    if request_type == "Navigation_Only":
        graph_type = "unweighted"
    else:
        graph_type = "weighted"

    # Ask if user wants comparison mode (only for Navigation_Only)
    compare_mode = False
    if request_type == "Navigation_Only":
        try:
            choice = input(
                "\n Run all search algorithms for comparison? (y/n): "
            ).strip().lower()
            compare_mode = (choice == "y")
        except EOFError:
            pass

    search_output = run_search(source, dest,
                               graph_type=graph_type,
                               compare=compare_mode)
    display_search_output(search_output, show_comparisons=compare_mode)

# — Step 6: Final Response —————
final_resp = build_final_response(

```

```

        request, ann_output, logic_output, csp_output, search_output
    )
    return final_resp

# -----
# Main loop
# -----

def main():
    """
    Main entry point.

    Runs an interactive CLI loop that lets the user submit
    multiple requests until they choose to exit.
    """

    print("\n" + "=" * 60)
    print(" SMART CAMPUS AI DECISION SUPPORT & AUTOMATION SYSTEM")
    print(" AL2002 – Artificial Intelligence Lab (Spring 2026)")
    print(" NUCES – Computer Science Department")
    print("=" * 60)
    print("Type 'exit' or 'quit' as name to stop the program.\n")

    session_count = 0

    while True:
        try:
            # — Collect structured input -----

```

```

request = collect_input()

# Check exit condition (handled inside collect_input via name)
name_check = request.get("name", "").strip().lower()
if name_check in ("exit", "quit"):
    print("\n[★] Thank you for using Smart Campus AI. Goodbye!\n")
    break

session_count += 1

print(f"\n[✓] Processing Request #{session_count} "
      f"(ID: {request['request_id']})...")

# — Execute pipeline —————
final_resp = execute_pipeline(request)

# — Display final response —————
display_final_response(final_resp)

# — Optional: save to JSON —————
try:
    save = input(" Save response to JSON file? (y/n): ").strip().lower()
    if save == "y":
        filename = f"response_{final_resp['request_id']}.json"
        export_response_json(final_resp, filename)
except EOFError:
    pass

```

```

# — Continue or exit —————
try:
    cont = input("\n Submit another request? (y/n): ").strip().lower()
    if cont != "y":
        print("\n[★] Thank you for using Smart Campus AI. Goodbye!\n")
        break
except EOFError:
    break

except KeyboardInterrupt:
    print("\n\n[!] Interrupted. Exiting Smart Campus AI.\n")
    break

except Exception as e:
    print(f"\n[!] An unexpected error occurred: {e}")
    print("\n Please try again.\n")

if __name__ == "__main__":
    main()

```

OUTPUTS

Navigation:

```
[+] C:\Users\jacob\Documents\Smart_Campus_Project\Smart_Campus_Python\main.py
[+] Running Search & Navigation Module...

Run all search algorithms for comparison? (y/n): y

-----
SEARCH & NAVIGATION RESULT
-----
Algorithm      : BFS
Path           : Hostel → Cafeteria → Science_Block → AI_Lab
Cost           : 3
Steps          : 3
Nodes Expanded: 7

Comparison with other algorithms:
Algorithm      Cost  Steps  Expanded  Path
-----
BFS            3    3      7  Hostel → Cafeteria → Science_Block → AI_Lab
DFS            7    7     10  Hostel → Parking → Main_Gate → Admin_Block → Student_Services → Library → Science_Block → AI_Lab
DLS            6    6     10  Hostel → Parking → Main_Gate → Admin_Block → Student_Services → Library → AI_Lab
IDS            3    3     27  Hostel → Cafeteria → Science_Block → AI_Lab
UCS            7    3      8  Hostel → Cafeteria → Science_Block → AI_Lab
BiBFS          3    3      5  Hostel → Cafeteria → Science_Block → AI_Lab
Greedy         3    3      4  Hostel → Cafeteria → Science_Block → AI_Lab
A*             7    3      4  Hostel → Cafeteria → Science_Block → AI_Lab
RBFS           7    3      3  Hostel → Cafeteria → Science_Block → AI_Lab
-----

SMART CAMPUS AI - FINAL RESPONSE

Request ID      : REQFDA7D3
Decision       : ✓ ACCEPTED
Algorithm      : BFS
Route          : Hostel → Cafeteria → Science_Block → AI_Lab
Route Cost     : 3
Steps          : 3

Best route from Hostel to AI_Lab found using BFS:
Hostel → Cafeteria → Science_Block → AI_Lab (Cost: 3, Steps: 3).
```

Booking and Scheduling

```
[✓] Processing Request #3 (ID: REQF007D3)...

-----
REQUEST ROUTER DECISION
-----
Request ID : REQF007D3
Pipeline  : Logic_KB → CSP
ANN       : No
Logic/KB  : Yes
CSP       : Yes
Search    : No
-----

[→] Running Logic / Knowledge Base Module...

-----
LOGIC / KNOWLEDGE BASE RESULT
-----
Status      : ALLOWED ✓
Reason      : Request is valid. Role 'instructor' is authorized for 'Access_Request'.
Explanation Chain:
  [OK] Role 'instructor' is authorized for category 'Access_Request'.
  [OK] Instructor Alina has access rights.
-----

[→] Running CSP Scheduler Module...

-----
CSP SCHEDULER RESULT
-----
Decision      : ACCEPTED ✓
Assigned Room : Admin_Block
Assigned Slot : 2
Destination   : Admin_Block
Notes         : Preferred slot 2 was successfully assigned.
-----



SMART CAMPUS AI — FINAL RESPONSE
    

Request ID : REQF007D3


```

8. Conclusion

This project successfully demonstrates the integration of:

- Search Algorithms
- Logic-Based Reasoning
- CSP
- Artificial Neural Networks

The system provides intelligent and efficient campus assistance through structured request handling.

